

Как работают компоненты и пропсы в Астро

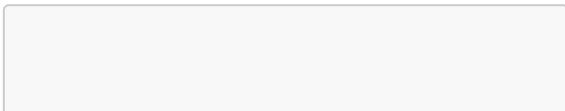
Оглавление

- Как работают компоненты и пропсы в Астро
 - Оглавление
- Проблема
- Цель
- Решение
 - Пропсы компонента
 - Компоненты
 - Структура компонента
 - Скрипт компонента
 - Шаблон компонента
 - Компонентный дизайн

- Слоты
- Именованные слоты
- Резервный контент для слотов
- Перенос слотов
- HTML-компоненты
- Дополнительно (глубже в кроличью нору)
 - Fragments
 - set:directives
 - Справочник директив шаблонов
 - Правила
 - Общие директивы
 - class:list
 - set:html
 - set:text
 - client:directives
 - client:load

- client:idle
 - timeout
 - client:visible
 - client:visible=
{{rootMargin}}
 - framework-components
 - dynamic-tags
 - passing-components-to-
mdx-content
 - requestIdleCallback
 - requestIdleCallback-
method
 - hydration
 - XSS
- [Материалы для статьи](#)

Проблема



передача информации между
файлами в Astro

Цель

- Разобраться с передачей пропсов
- Лучшие способы передачи
- Разбор компонентов в пропсах

Решение

Пропсы компонента

Компонент **Astro** может определять и принимать входные параметры — пропсы. Эти пропсы затем становятся доступными шаблону компонента для рендеринга HTML. Пропсы доступны в

глобальном объекте ***Astro.props*** в вашем скрипте.

Вот пример компонента, который получает пропсы **greeting** и **name**.

Обратите внимание, что получаемые входные параметры деструктурированы из глобального объекта ***Astro.props***.

```
---  
// Применение:  
<GreetingHeadline  
  greeting="Привет"  
  name="Партнёр" />  
const { greeting, name } =  
  Astro.props;  
---  
<h2>{greeting}, {name}!  
</h2>
```

Этот компонент при импорте и визуализации в других компонентах **Astro**, макетах или страницах может передавать эти параметры в качестве атрибутов:

```
---
import GreetingHeadline
from
'./GreetingHeadline.astro';
const name = "Astro"
---
<h1>Поздравительная
открытка</h1>
<GreetingHeadline
greeting="Привет" name=
{name} />
<p>Надеюсь, у вас чудесный
день!</p>
```

Вы также можете определить свои пропсы с помощью **TypeScript** с интерфейсом типа **Props**. Astro автоматически подхватит интерфейс **Props** в вашем блоке метаданных и выдаст предупреждения/ошибки типа. Этим входным параметрам также могут быть присвоены значения по умолчанию при деструктуризации из ***Astro.props***.

```
---  
interface Props {  
  name: string;  
  greeting?: string;  
}  
  
const { greeting =  
  "Привет", name } =  
  Astro.props;  
---  
<h2>{greeting}, {name}!  
</h2>
```

Пропсам компонента могут быть присвоены значения по умолчанию, которые можно использовать, если они не указаны.

```
---  
const { greeting =  
  "Привет", name =  
  "Астронавт" } =  
  Astro.props;  
---  
<h2>{greeting}, {name}!  
</h2>
```

Компоненты

Структура компонента

Компонент **Astro** состоит из двух основных частей: Скрипта компонента и Шаблона компонента. Каждая часть выполняет свою работу, но вместе они обеспечивают структуру, которая одновременно проста в использовании и достаточно выразительна, чтобы справиться с тем, что вы захотите создать.

```
---  
// Скрипт компонента  
(JavaScript)  
---  
<!-- Шаблон компонента  
(HTML + выражения JS) -->
```

Скрипт компонента

Astro использует блок кода (**---**) для идентификации скрипта в вашем

компоненте **Astro**. Если вы когда-либо писали Markdown раньше, возможно, вы уже знакомы с похожей концепцией, называемой метаданные. Идея **Astro** о скрипте компонента была непосредственно вдохновлена этой концепцией.

Вы можете использовать скрипт компонента для написания любого кода **JavaScript**, необходимого для визуализации шаблона. Это может включать в себя:

импорт других компонентов **Astro** импорт компонентов из других фреймворков, таких как **React** импорт данных, например файла **JSON** получение контента из **API** или базы данных создание переменных, на которые вы будете ссылаться в своем шаблоне

```
---  
import SomeAstroComponent  
from  
'../components/SomeAstroCom  
ponent.astro';  
import SomeReactComponent  
from  
'../components/SomeReactCom  
ponent.jsx';  
import someData from  
'../data/pokemon.json';  
  
// Доступ к пропсам  
компонента, таким как `title="Привет, мир" />`  
const {title} =  
Astro.props;  
  
// Получение внешних  
данных, даже из приватного  
API или базы данных  
const data = await  
fetch('SOME_SECRET_API_URL/
```

```
users').then(r =>
  r.json());
---
<!-- Здесь ваш шаблон! -->
```

Блок кода предназначен для того, чтобы гарантировать, что написанный в нём **JavaScript** остаётся «изолированным». Он не попадёт в ваше фронтенд-приложение и не окажется в руках пользователя. Здесь можно безопасно писать код, который является ресурсоёмким или чувствительным (например, подключение к вашей приватной базе данных), не беспокоясь о том, что он когда-либо попадёт в браузер пользователя.

Шаблон компонента

Шаблон компонента следует за блоком кода и определяет вывод **HTML** вашего

компонента.

Если вы напишете здесь простой **HTML**, ваш компонент будет отображать этот **HTML** на любой странице Astro, в которую он импортирован и используется.

Однако, синтаксис **Astro** так же поддерживает **JavaScript** выражения, теги *<style> u <script>* **Astro**, импортированные компоненты, и специальные директивы **Astro** . Данные и значения, определённые в скрипте компонента, можно использовать в шаблоне компонента для создания динамически создаваемого **HTML**.

```
---  
// Здесь ваш скрипт  
компонента!  
import Banner from
```

```
'../components/Banner.astro'
';
import Avatar from
'../components/Avatar.astro'
';
import
ReactPokemonComponent from
'../components/ReactPokemon
Component.jsx';
const myFavoritePokemon =
[/ * ... */];
const { title } =
Astro.props;
---
<!-- Поддерживаются HTML
комментарии! -->
{ /* Синтаксис комментариев
JS также действителен! =>
скорее jsx */ }

<Banner />
<h1>Привет, мир!</h1>

<!-- Используем пропсы и
```

другие переменные из
скрипта компонента: -->

```
<p>{title}</p>
```

<!-- Осуществляем
отложенный рендеринг
компонента с резервным
контентом для загрузки: -->

```
<Avatar server:defer>  
  <svg slot="fallback"  
class="generic-avatar"  
transition:name="avatar">..  
.</svg>  
</Avatar>
```

<!-- Включаем другие
компоненты
пользовательского
интерфейса с помощью
директивы `client`: для
гидратации: -->

```
<ReactPokemonComponent  
client:visible />
```

```
<!-- Комбинируем HTML с  
выражениями JavaScript, как  
в JSX: -->
```

```
<ul>
```

```
{myFavoritePokemon.map((dat  
a) => <li>{data.name}  
</li>)}  
</ul>
```

```
<!-- Используем директиву  
шаблона для создания имён  
классов из нескольких строк  
или даже объектов! -->
```

```
<p class:list={["add",  
"dynamic", {classNames:  
true}]} />
```

Компонентный дизайн

Компоненты спроектированы так, чтобы их можно было повторно использовать и

компоновать. Вы можете использовать компоненты внутри других компонентов для создания более продвинутого пользовательского интерфейса.

Например, компонент **Button** можно использовать для создания компонента **ButtonGroup**:

```
---
import Button from
'./Button.astro';
---
<div>
  <Button title="Кнопка 1"
/>
  <Button title="Кнопка 2"
/>
  <Button title="Кнопка 3"
/>
</div>
```

Слоты

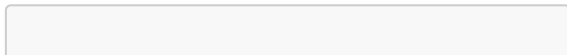
`<slot />` это заполнитель для внешнего **HTML**-содержимого, позволяющий вставлять дочерние элементы из других файлов в шаблон компонента.

По умолчанию все дочерние элементы, переданные компоненту, будут отображаться в его **`<slot />`**.

Заметка

В отличие от пропсов, которые являются атрибутами, передаваемыми компоненту **Astro**, доступными для использования во всем вашем компоненте с помощью ***Astro.props***, слоты отображают дочерние элементы **HTML** там, где они написаны.

- `src/components/Wrapper.astro`



```
---
import Header from
'./Header.astro';
import Logo from
'./Logo.astro';
import Footer from
'./Footer.astro';

const { title } =
Astro.props
---
<div id="content-wrapper">
  <Header />
  <Logo />
  <h1>{title}</h1>
  <slot />  <!-- дочерний
элемент будет здесь -->
  <Footer />
</div>
```

- src/pages/fred.astro

```
---
import Wrapper from
'../components/Wrapper.astro';
---

<Wrapper title="Fred's
Page">
  <h2>Всё о Фреде</h2>
  <p>Вот немного информации
о Фреде.</p>
</Wrapper>
```

Этот шаблон является основой компонента макета **Astro**: целая страница **HTML**-контента может быть «обёрнута» тегами и передана компоненту для отрисовки внутри определённых в нём общих элементов страницы.

Именованные слоты

Компонент **Astro** также может иметь именованные слоты. Это позволяет передавать в местоположение слота только элементы **HTML** с соответствующим именем слота.

Слоты именуются с помощью атрибута ***name***:

- `src/components/Wrapper.astro`

```
---  
import Header from  
  './Header.astro';  
import Logo from  
  './Logo.astro';  
import Footer from  
  './Footer.astro';  
  
const { title } =  
  Astro.props  
---
```

```
<div id="content-wrapper">
  <Header />
  <!-- дочерние элементы с
атрибутом `slot="after-
header"` будут находиться
здесь -->
  <slot name="after-
header"/>
  <Logo />
  <h1>{title}</h1>
  <!-- дочерние элементы
без `slot`, или с
`slot="default"` будут
находиться здесь -->
  <slot />
  <Footer />
  <!-- дочерние элементы с
атрибутом `slot="after-
footer"` будут находиться
здесь -->
  <slot name="after-
footer"/>
</div>
```

Чтобы внедрить **HTML**-контент в определённый слот, используйте атрибут ***slot*** в любом дочернем элементе, чтобы указать имя слота. Все остальные дочерние элементы компонента будут вставлены в стандартный (безымянный) ***<slot />***.

- `src/pages/fred.astro`

```
---
import Wrapper from
'../components/Wrapper.astro';
---
<Wrapper title="Страница
Фреда">
  
  <h2>Всё о Фреде</h2>
  <p>Вот немного информации
```

```
о Фреде.</p>
  <p slot="after-
footer">Авторское право
2022</p>
</Wrapper>
```

- *** Совет Используйте атрибут ***slot="my-slot"*** в дочернем элементе, который вы хотите передать в соответствующий заполнитель ***<slot name="my-slot" />*** в вашем компоненте.

Чтобы передать несколько HTML-элементов в заполнитель ***<slot/>*** компонента без обёртывающего ***<div>***, используйте атрибут ***slot=""*** на компоненте ***<Fragment/>***

- `src/components/CustomTable.astro`


```
---  
//  Создание  
пользовательской таблицы с  
именованными заполнителями  
для заголовка и тела  
контента  
---  
<table class="bg-white">  
  <thead class="sticky top-  
0 bg-white"><slot  
name="header" /></thead>  
  <tbody class="[_tr:nth-  
child(odd)]:bg-gray-100">  
    <slot name="body" />  
  </tbody>  
</table>
```

Вставьте несколько строк и столбцов
HTML-контента с помощью атрибута
slot="", чтобы указать содержимое

"header" и "body". Отдельные элементы **HTML** также можно стилизовать:

- `src/components/StockTable.astro`

```
---
import CustomTable from
'./CustomTable.astro';
---
<CustomTable>
  <Fragment slot="header">
    <!-- передаём заголовок
    таблицы -->
    <tr><th>Название
    продукта</th><th>Единицы на
    складе</th></tr>
  </Fragment>

  <Fragment slot="body">
    <!-- передаём тело таблицы
    -->
    <tr><td>Шлёпанцы</td>
    <td>64</td></tr>
```

```
<tr><td>Ботинки</td>
<td>32</td></tr>
<tr><td>Кроссовки</td>
<td class="text-red-
500">0</td></tr>
</Fragment>
</CustomTable>
```

Обратите внимание, что именованные слоты должны быть непосредственными дочерними элементами компонента. Вы не можете передавать именованные слоты через вложенные элементы.

- *** Совет

Именованные слоты также можно передавать в [компоненты UI фреймворков!](#)

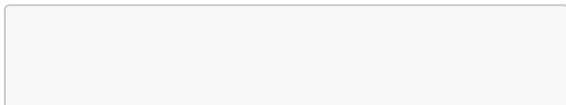
- *** Заметка

Невозможно динамически генерировать имя слота **Astro**, например, внутри функции **map**. Если эта функция необходима в компонентах UI-фреймворка, лучше всего генерировать эти динамические слоты внутри самого фреймворка.

Резервный контент для слотов

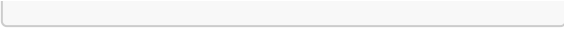
Слоты также могут отображать резервный контент. Когда в слот не передаются соответствующие дочерние элементы, элемент **<slot />** отобразит свои собственные дочерние элементы-заполнители.

- `src/components/Wrapper.astro`



```
---
import Header from
'./Header.astro';
import Logo from
'./Logo.astro';
import Footer from
'./Footer.astro';

const { title } =
Astro.props
---
<div id="content-wrapper">
  <Header />
  <Logo />
  <h1>{title}</h1>
  <slot>
    <p>Это резервное
содержимое, если в слот не
передан дочерний
элемент</p>
  </slot>
  <Footer />
</div>
```



Резервный контент будет отображаться только в том случае, если нет соответствующих элементов с атрибутом ***slot="name"***, передаваемых в именованный слот.

Astro передаст пустой слот, если элемент слота существует, но не содержит контента для передачи. Резервный контент нельзя использовать в качестве значения по умолчанию, когда передаётся пустой слот. Резервный контент отображается только тогда, когда элемент слота не найден.

- как по мне почти бессмысленно лучше уж рухнет если что... разве что если отображать и обрабатывать с базы данных... и вывести надпись данных нет но тут

астро как по мне проигрывает
next.js

Перенос слотов

Слоты можно передавать другим компонентам. Например, при создании вложенных макетов:

- `src/layouts/BaseLayout.astro`

```
---  
---  
<html lang="en">  
  <head>  
    <meta charset="utf-8"  
  />  
    <link rel="icon"  
type="image/svg+xml"  
href="/favicon.svg" />  
    <meta name="viewport"  
content="width=device-
```

```
width" />
    <meta name="generator"
content={Astro.generator}
/>
    <slot name="head"/>
</head>
<body>
    <slot />
</body>
</html>
```

- src/layouts/HomeLayout.astro

```
---
import BaseLayout from
"./BaseLayout.astro";
---
<BaseLayout>
    <slot name="head"
slot="head"/>
    <slot />
</BaseLayout>
```


- Результат

```
<html lang="en">
  <head>
    <meta charset="utf-8"
  />
    <link rel="icon"
type="image/svg+xml"
href="/favicon.svg" />
    <meta name="viewport"
content="width=device-
width" />
    <meta name="generator"
content="Astro vX.Y.Z" />

    <!-- Слот head пуст -->

  </head>
  <body>
    <!-- Слот по умолчанию
пуст -->
```

```
</body>  
</html>
```

- *** Моя Заметка Может быть очень полезно для SEO
- *** Заметка

Именованные слоты могут быть переданы другому компоненту, используя атрибуты ***name u slot*** в теге ***<slot />***

Теперь содержимое стандартного (безымянного) слота и слота head, переданных в HomeLayout, будет перенесено в родительский компонент BaseLayout.

- src/pages/index.astro

```
---
import HomeLayout from
"../layouts/HomeLayout.astro";
---
<HomeLayout>
  <title
slot="head">Astro</title>
  <h1>Astro</h1>
</HomeLayout>
```

- Результат

```
<html lang="en">
  <head>
    <meta charset="utf-8"
/>
    <link rel="icon"
type="image/svg+xml"
href="/favicon.svg" />
    <meta name="viewport"
```

```
content="width=device-  
width" />  
    <meta name="generator"  
content="Astro vX.Y.Z" />  
  
    <!-- Title "Astro"  
попадает сюда благодаря  
перенаправлению слотов -->  
    <title>Astro</title>  
  
</head>  
<body>  
    <!-- Заголовок "Astro"  
попадает сюда как основной  
КОНТЕНТ -->  
    <h1>Astro</h1>  
    </body>  
</html>
```

HTML-компоненты

Astro поддерживает импорт и использование *.html* файлов в качестве компонентов или размещение этих файлов в подкаталоге *src/pages/* в качестве страниц. Возможно, вам захочется использовать **HTML**-компоненты, если вы повторно используете код с существующего сайта, созданного без фреймворка, или если вы хотите убедиться, что ваш компонент не имеет динамических функций.

HTML-компоненты должны содержать только валидный **HTML**, поэтому они лишены ключевых возможностей компонентов **Astro**:

Они не поддерживают метаданные, импорт на стороне сервера или динамические выражения. Любые теги *<script>* остаются неупакованными, и

обрабатываются, как если бы у них был атрибут **is:inline**. Они могут только ссылаться на ресурсы, которые находятся в папке **public/**.

- *** Заметка

В **HTML**-компоненте элемент **<slot />** будет работать так же, как и в компоненте **Astro**. Чтобы использовать **HTML Web Component Slot**, добавьте атрибут **is:inline** к вашему элементу **<slot>**.

Дополнительно (глубже в кроличью нору)

Fragments

Astro поддерживает **<> </>** нотацию и предоставляет встроенный компонент. Этот компонент может быть полезен для

предотвращения использования элементов-обёрток при добавлении `set:*directives` для внедрения **HTML**-строки.

В следующем примере текст абзаца визуализируется с использованием компонента:

```
---  
const htmlString = '<p>Raw  
HTML content</p>';  
---  
<Fragment set:html=  
{htmlString} />
```

set:directives

Справочник директив шаблонов

Директивы шаблона — это особый вид **HTML**-атрибута, доступный внутри любого шаблона компонента **Astro** (**.astro**), и некоторые из них также могут использоваться в **.mdx**.

Директивы шаблона используются для управления поведением элемента или компонента. Директива шаблона может включать какую-либо функцию компилятора, упрощающую вашу работу (например, использование **class:list** вместо **class**). Или директива может указывать компилятору **Astro** на необходимость выполнения каких-либо особых действий с этим компонентом (например, [гидратации](#) с помощью **client:load**).

В этой части описываются все директивы шаблонов, доступные вам в **Astro**, и

принцип их работы.

Правила

Чтобы шаблонная директива была действительной, она должна:

Включите двоеточие :в его имя, используя форму **X:Y**(например: **client:load**). Быть видимым для компилятора (например: *****<X {...attr}>*****не будет работать, если **attr** содержит директиву). Некоторые директивы шаблона (но не все) могут принимать пользовательское значение:

<X client:load />(не имеет значения) **<X class:list={{['some-css-class']}} />**

(принимает массив) Директива шаблона никогда не включается напрямую в конечный **HTML**-вывод компонента.

Общие директивы

class:list

class:list={...} Принимает массив значений класса и преобразует их в строку класса. Работает на основе популярной вспомогательной библиотеки [clsx от @lukeed](#).

class:list принимает массив из нескольких возможных типов значений:

string: Добавлено к элементу class **Object:** Все ключи истины добавляются к элементу class **Array:** сплюснутый **false, null, или undefined:** пропущено

```
<!-- This -->
<span class:list={[ 'hello
goodbye', { world: true },
[ 'friend' ] ]} />
<!-- Becomes -->
```

```
<span class="hello goodbye  
world friend"></span>
```

set:html

set:html={string} внедряет **HTML**-строку в элемент, аналогично настройке ***el.innerHTML***.

Astro не экранирует значение автоматически! Убедитесь, что вы доверяете этому значению, или экранировали его вручную перед передачей в шаблон. Если вы забудете сделать это, вы рискуете стать жертвой [межсайтового скриптинга \(XSS\)](#).

```
---  
const rawHTMLString =  
"Hello
```

```

<strong>World</strong>"
---
<h1>{rawHTMLString}</h1>
  <!-- Output: <h1>Hello
    &lt;strong&gt;World&lt;/strong&gt;</h1> -->
  <h1 set:html=
    {rawHTMLString} />
    <!-- Output: <h1>Hello
      <strong>World</strong></h1>
    -->

```

Вы также можете использовать **set:html** ,
****<Fragment>**** чтобы избежать
 добавления ненужного элемента-обёртки.
 Это может быть особенно полезно при
 получении **HTML** из **CMS**.

```

---
const cmsContent = await
  fetchHTMLFromMyCMS();

```

```
---  
<Fragment set:html=  
{cmsContent}>
```

****set:html={Promise<string>}**** вставляет **HTML**-строку в элемент, заключенный в **Promise**.

Это можно использовать для внедрения **HTML**-кода, хранящегося снаружи, например, в базе данных.

```
---  
import api from  
'../db/api.js';  
---  
<article set:html=  
{api.getArticle(Astro.props  
.id)}></article>
```

set:html=

{Promise<Response>} внедряет ответ в элемент.

Это особенно полезно при использовании **fetch()**. Например, при извлечении старых записей из предыдущего генератора статических сайтов.

```
<article set:html=
{fetch('http://example/old-
posts/making-soup.html')}>
</article>
```

set:html Может использоваться с любым тегом и не обязательно включать HTML. Например, используйте с **JSON.stringify()** с **<script>** тегом, чтобы добавить схему **JSON-LD** на страницу.

```
<script
type="application/ld+json"
set:html={JSON.stringify({
  "@context":
  "https://schema.org/",
  "@type": "Person",
  name: "Houston",
  hasOccupation: {
    "@type": "Occupation",
    name: "Astronaut"
  }
})}/>
```

set:text

set:text={string} Вставляет текстовую строку в элемент, аналогично настройке **el.innerText**. В отличие от **set:html**, string передаваемое значение автоматически экранируется **Astro**.

Это эквивалентно простой передаче переменной напрямую в выражение шаблона (например: **<div>{someText}</div>**), поэтому эта директива обычно не используется.

client:directives

Эти директивы управляют тем, как [компоненты UI Framework](#) размещаются на странице.

По умолчанию [компонент UI Framework](#) не обрабатывается на клиенте. Если **client:*** директива не указана, его **HTML**-код отображается на странице без **JavaScript**.

Директиву **client** можно использовать только в [компоненте UI-фреймворка](#), который напрямую импортируется в **.astro** компонент. Директивы **Hydration** не поддерживаются при использовании

динамических тегов и пользовательских компонентов, передаваемых через `components` свойство .

client:load

- Приоритет: Высокий
- Полезно для: мгновенно видимых элементов пользовательского интерфейса, которые должны стать интерактивными как можно скорее. Загружайте и гидратируйте компонент **JavaScript** немедленно при загрузке страницы.

```
<BuyButton client:load />
```

client:idle

- Приоритет: Средний

- Полезно для: низкоприоритетных элементов пользовательского интерфейса, которые не требуют немедленного взаимодействия. Загрузите и обновите **JavaScript**-компонент после завершения первоначальной загрузки страницы и [requestIdleCallback](#) срабатывания события. Если ваш браузер не поддерживает , будет использоваться событие [requestIdleCallback](#) документа **.load**

```
<ShowHideButton client:idle  
/>
```

timeout

Добавлено в: astro@4.15.0

Максимальное время ожидания (в **миллисекундах**) перед загрузкой компонента, даже если начальная загрузка страницы еще не завершена.

Это позволяет передавать значение параметра `timeout` из `requestIdleCallback()` спецификации . Это означает, что вы можете отложить гидратацию для элементов пользовательского интерфейса с более низким приоритетом, обеспечивая больший контроль и гарантируя интерактивность элемента в течение заданного периода времени.

```
<ShowHideButton  
  client:idle={{timeout:  
    500}} />
```

client:visible

- Приоритет: низкий
- Полезно для: низкоприоритетных элементов пользовательского интерфейса, которые либо находятся далеко внизу страницы («ниже сгиба»), либо требуют настолько много ресурсов для загрузки, что вы бы предпочли не загружать их вообще, если пользователь никогда не увидит этот элемент. Загружайте и гидратируйте **JavaScript**-код компонента после того, как он попадает в область просмотра пользователя. **IntersectionObserver** Для отслеживания видимости используется внутренний механизм.

```
<HeavyImageCarousel  
  client:visible />
```

client:visible={{rootMargin}}

Добавлено в: astro@4.1.0

При необходимости значение **rootMargin** может быть передано базовому **IntersectionObserver**. Если **rootMargin** указано, **JavaScript**-компонент будет гидратироваться, когда заданное поле (в пикселях) вокруг компонента попадает в область просмотра, а не сам компонент.

```
<HeavyImageCarousel  
  client:visible=  
    {{rootMargin: "200px"}} />
```

Указание **rootMargin** значения может сократить сдвиги макета (CLS), дать компоненту больше времени для загрузки

при более медленном интернет-соединении и сделать компоненты интерактивными быстрее, повысив стабильность и скорость реагирования страницы.

client:media

- Приоритет: низкий
- Полезно для: переключателей боковой панели и других элементов, которые могут быть видны только на экранах определенных размеров.

client:media={string} загружает и обновляет компонент **JavaScript** после выполнения определенного медиа-запроса **CSS**.

Примечание

Если компонент уже скрыт и показан с помощью медиа-запроса в вашем **CSS**, то может быть проще просто использовать его **client:visible** и не передавать тот же медиа-запрос в директиву.

```
<SidebarToggle  
  client:media="(max-width:  
  50em)" />
```

client:only

client:only={string} Пропускает рендеринг **HTML** на сервере и отображает его только на клиенте. Он действует аналогично: ***client:load*** загружает, отображает и обновляет компонент сразу после загрузки страницы.

Необходимо передать корректную платформу компонента в качестве значения! Поскольку **Astro** не запускает компонент во время сборки на сервере, **Astro** не знает, какую платформу использует ваш компонент, пока вы не укажете её явно.

```
<SomeReactComponent  
  client:only="react" />  
<SomePreactComponent  
  client:only="preact" />  
<SomeSvelteComponent  
  client:only="svelte" />  
<SomeVueComponent  
  client:only="vue" />  
<SomeSolidComponent  
  client:only="solid-js" />
```


Отображение загружаемого содержимого

Для компонентов, которые отображаются только на клиенте, также можно отображать резервный контент во время загрузки. Используйте ***slot="fallback"*** этот параметр для любого дочернего элемента, чтобы создать контент, который будет отображаться только до тех пор, пока ваш клиентский компонент не станет доступен:

```
<ClientComponent
  client:only="vue">
  <div
    slot="fallback">Loading</div>
</ClientComponent>
```

Индивидуальные клиентские директивы

Начиная с версии Astro 2.6.0 интеграции также позволяют добавлять пользовательские ***client:**** директивы для изменения того, как и когда следует гидратировать компоненты.

Посетите часть [addClientDirectiveAPI](#) , чтобы узнать больше о создании пользовательской клиентской директивы.

Директивы сервера

Эти директивы управляют отображением компонентов серверного острова.

server:defer

Директива ***server:defer*** преобразует компонент в серверный остров, заставляя

его отображаться по требованию, вне области отображения остальной части страницы.

Подробнее об использовании компонентов серверного острова [см.](#)

```
<Avatar server:defer />
```

Директивы по скрипту и стилю

Эти директивы можно использовать только в **HTML** `<script>` и `<style>` тегах для управления тем, как клиентский **JavaScript** и **CSS** обрабатываются на странице.

is:global

По умолчанию **Astro** автоматически применяет `<style>` **CSS**-правила к компоненту. Вы можете отключить это поведение с помощью ***is:global*** директивы.

is:global При включении компонента содержимое тега `<style>` применяется глобально на странице. Это отключает систему **CSS**-областей действия **Astro**. Это эквивалентно заключению всех селекторов в `<style>` тег с ***:global()***.

Вы можете объединить `<style>` и `<style is:global>` в одном компоненте, чтобы создать некоторые глобальные правила стиля, сохраняя при этом область действия большей части **CSS** вашего компонента.

Более подробную информацию о работе глобальных стилей см . на странице

«Стилизация и CSS» .

```
<style is:global>  
  body a { color: red; }  
</style>
```

is:inline

По умолчанию **Astro** обрабатывает, оптимизирует и объединяет все теги `<script>` и `<style>` которые видит на странице. Вы можете отключить это поведение с помощью ***is:inline*** директивы.

is:inline **Astro** указывает оставить тег `<script>` или `<style>` как есть в конечном **HTML**-коде. Содержимое не будет обработано, оптимизировано или упаковано. Это ограничивает некоторые функции **Astro**, такие как импорт пакета

npm или использование языка компиляции в **CSS**, например, **Sass**.

Директива ***is:inline*** означает, что `<style>` и `<script>` теги:

- Не будет объединено во внешний файл. Это означает, что атрибуты, например ***defer***, управляющие загрузкой внешнего файла, не будут иметь никакого эффекта.
- Дедупликация не производится — элемент будет появляться столько раз, сколько он отображается.
- Ссылки ***// import*** не будут разрешены относительно файла ***.@import url().astro***
- Будет отображен в конечном выходном **HTML**-коде именно там,

где он был создан.

- Стили будут глобальными и не будут ограничены компонентом.
- *** Осторожность

Эта `is:inline` директива подразумевается всякий раз, когда в теге используется любой атрибут, кроме `.src`. Единственным исключением является использование директивы в теге, которое не подразумевает автоматически.

```
<script>  
<style>define:vars<style>is  
:inline
```

```
<style is:inline>  
/* inline: relative & npm
```

```
package imports are not
supported. */
  @import '/assets/some-
public-styles.css';
  span { color: green; }
</style>
```

```
<script is:inline>
  /* inline: relative & npm
package imports are not
supported. */
  console.log('I am inlined
right here in the final
output HTML.');
```

Посмотрите, как работают **клиентские скрипты** в компонентах Astro.

define:vars

define:vars={...} Может передавать серверные переменные из ***frontmatter*** вашего компонента в клиент `<script>` или `<style>` теги. Поддерживаются любые **JSON**-сериализуемые переменные ***frontmatter***, в том числе **props** переданные в ваш компонент через ***Astro.props***. Значения сериализуются с помощью ***JSON.stringify()***.

```
---
const foregroundColor =
  "rgb(221 243 228)";
const backgroundColor =
  "rgb(24 121 78)";
const message = "Astro is
  awesome!";
---
<style define:vars={{
  textColor: foregroundColor,
  backgroundColor }}>
  h1 {
```

```
        background-color: var(-
-background-color);
        color: var(--
text-color);
    }
</style>

<script define:vars={{
message }}>
    alert(message);
</script>
```

- *** Осторожность

Использование ***define:vars*** тега `<script>` подразумевает ***is:inline*** директиву, что означает, что ваши скрипты не будут объединены, а будут встроены непосредственно в **HTML**.

Это связано с тем, что когда **Astro** объединяет скрипт, он включает и

запускает его один раз, даже если вы включаете компонент, содержащий скрипт, несколько раз на одной странице.

define:vars требует повторного запуска скрипта с каждым набором значений, поэтому вместо этого **Astro** создает встроенный скрипт.

Для скриптов попробуйте передавать переменные в скрипты вручную .

Расширенные директивы

is:raw

is:raw указывает компилятору **Astro** обрабатывать все дочерние элементы этого элемента как текст. Это означает, что весь специальный синтаксис шаблонов **Astro** будет игнорироваться внутри этого компонента.

Например, если у вас есть пользовательский компонент **Katex**, который преобразует текст в **HTML**, вы можете попросить пользователей сделать следующее:

```
import Katex from
'../components/Katex.astro';
```

Some conflicting {syntax} here

framework-components

! NF!!!!

dynamic-tags

passing-components-to-mdx-content

requestIdleCallback

requestIdleCallback-method

addclientdirective-option

server-islands

global-styles

defer

client-side-scripts

hydration

Термин "гидратироваться" (hydrate / hydration) в контексте Astro и современных фреймворков (таких как React, Vue, Svelte) означает процесс превращения статического, неинтерактивного HTML-кода, который был сгенерирован на сервере, в полностью интерактивный, работающий клиентский компонент с помощью JavaScript.

Вот подробное объяснение того, что подразумевается под этим словом в приведенных вами примерах:

Что такое "гидратация" (Hydration)? В мире разработки веб-приложений гидратация — это ключевой этап, следующий за серверным рендерингом (SSR) или генерацией статических сайтов (SSG).

Представьте, что вы строите дом. Сначала вы возводите каркас и стены (это статический HTML, который виден сразу). Затем вы подключаете электричество, водопровод и устанавливаете розетки и выключатели (это JavaScript, который делает дом функциональным).

Гидратация — это процесс подключения "электричества" к вашему HTML.

Как это работает в примере с Astro

В вашем примере: `<HeavyImageCarousel client:visible={{rootMargin: "200px"}} />`

На сервере: Когда пользователь запрашивает страницу, Astro отправляет готовый, но неактивный HTML-код для `HeavyImageCarousel`. Пользователь видит изображение-заглушку или статический макет карусели сразу, но кнопки "вперед/назад" пока не работают. JavaScript, необходимый для их работы, еще не загружен или не запущен.

На клиенте (в браузере): Когда браузер пользователя загружает JavaScript-файл для этого компонента и видит директиву `client:visible`, он начинает процесс гидратации.

Гидратация: JavaScript берет существующий HTML, "прикрепляет" к нему все обработчики событий

(слушатели кликов, свайпов) и инициализирует внутреннее состояние компонента.

Теперь карусель становится полностью функциональной и интерактивной.

Что означает `client:visible` и `rootMargin` в этом контексте

Директива `client:visible` указывает Astro отложить процесс гидратации до тех пор, пока компонент не попадет в поле зрения пользователя (вьюпорт). Это техника называется "ленивой гидратацией" (`lazy hydration`) или "частичной гидратацией" (`partial hydration`).

Слово "гидратироваться" здесь означает:

"Запустить и применить клиентский JavaScript к этому компоненту, чтобы

сделать его интерактивным".

Пример с rootMargin:

`<HeavyImageCarousel client:visible />`:

Компонент гидратируется (становится активным), когда его верхний край достигает видимой части экрана.

`<HeavyImageCarousel client:visible=`

`{{rootMargin: "200px"}} />`: Компонент

гидратируется (становится активным), когда до его появления на экране остается еще 200 пикселей прокрутки.

Зачем это нужно?

Как указано в вашем тексте, это делается для оптимизации:

Уменьшение CLS (Cumulative Layout Shift):

Компонент загружается заранее и не

вызывает резких "прыжков" страницы при появлении.

Ускорение загрузки страницы: Браузеру не нужно загружать и запускать тяжелый JavaScript сразу для всех компонентов страницы. Он гидратирует только те, которые скоро понадобятся пользователю.

По сути, "гидратироваться" — это технический термин, описывающий оживление статической веб-страницы с помощью JavaScript.

XSS

межсайтовый скриптинг

Материалы для статьи

[Астро документация](#)

[set:directives](#)

[Astro-syntax](#)

[XSS](#)

[Framework-components](#)

[dynamic-tags](#)

[passing-components-to-mdx-content](#)

[requestIdleCallback](#)

[requestidlecallback-method](#)

[addclientdirective-option](#)

[server-islands](#)

[global-styles](#)

[defer](#)

client-side-scripts